VILAG

Microsoft Teams Agent

Geliştirme Kılavuzu v2

RAG | Planner | Human-in-the-Loop

Bitirme Projesi

VILAG Monorepo üzerinde geliştirilecektir

İçindekiler

1. Projeye Genel Bakış
2. VILAG Mimarisi
3. Geliştirme Yol Haritası
4. Ön Koşul: pause/resume Mekanizması
5. Modül 1: RAG (Retrieval-Augmented Generation)
6. Modül 2: Planner (Planlama Katmanı)
7. Modül 3: Human-in-the-Loop
8. Modüllerin Entegrasyonu
9. Teams Senaryo Veritabanı
10. Test Stratejisi
11. Git Workflow
12. Zaman Planlaması

1. Projeye Genel Bakış

Bu kılavuz, VILAG projesine Microsoft Teams odaklı üç yeni modül eklemeniz için adım adım bir rehberdir. VILAG, UI-TARS Desktop projesinden esinlenerek sıfırdan yazılmış, daha temiz ve sadeleştirilmiş bir GUI Agent platformudur.

1.1. VILAG Nedir?

VILAG, web tabanlı uygulamaları doğrudan tarayıcı arayüzü üzerinden otonom olarak yönetebilen bir GUI Agent platformudur. Playwright ile tarayıcı kontrolü ve NutJS ile masaüstü kontrolü yapabilir. Electron + React ile geliştirilmiş bir masaüstü uygulamasıdır.

1.2. Ne Ekliyoruz?

- **RAG:** Teams iş akışlarını bir veritabanında tutup modele referans bilgi olarak sunma
- **Planner:** Karmaşık kullanıcı komutlarını alt görevlere bölme
- **Human-in-the-Loop:** Kritik adımlarda kullanıcıdan onay isteme

2. VILAG Mimarisi

Yeni modülleri nereye entegre edeceğinizi anlamak için VILAG'ın yapısını iyi bilmeniz gerekir.

2.1. Proje Yapısı

```
vilag-monorepo/
  apps/vilag-desktop/       → Electron masaüstü uygulaması
  packages/
    vilag-sdk/              → GUIAgent + UITarsModel (ana döngü)
    vilag-action-parser/    → Model çıktısını aksiyonlara parse eder
    vilag-browser-operator/ → Playwright tabanlı tarayıcı kontrolü
    vilag-desktop-operator/ → NutJS tabanlı masaüstü kontrolü
    vilag-electron-ipc/     → Electron IPC iletişimi
    vilag-logger/           → Renkli, timestamp'li loglama
    vilag-shared/           → Paylaşılan tipler ve sabitler
```

2.2. Ana Bileşenler

Bileşen	Dosya	Görev
GUIAgent	vilag-sdk/src/GUIAgent.ts	Ana orkestratör. Screenshot → Model → Execute döngüsü.
UITarsModel	vilag-sdk/src/Model.ts	OpenAI uyumlu API ile model iletişimi. Sadece Chat Completions.
Operator	vilag-sdk/src/types.ts	Soyut arayüz: screenshot() ve execute() metodları.
BrowserOperator	vilag-browser-operator/src/index.ts	Playwright ile tarayıcı kontrolü. Teams için bu kullanılacak.
NutJSOperator	vilag-desktop-operator/src/index.ts	Masaüstü fare/klavye kontrolü.
Action Parser	vilag-action-parser/src/index.ts	Model prediction'ını parse eder (v1.0, v1.5, doubao).
Logger	vilag-logger/src/index.ts	Renkli, formatlı log sistemi.

2.3. VILAG GUIAgent Akışı

VILAG'ın run() metodu şu döngüyü çalıştırır:

1. Kullanıcı instruction verir (main.ts üzerinden)
2. System prompt + instruction mesaj dizisine eklenir
3. Operator'dan screenshot alınır (base64 + scaleFactor)
4. Son ekran görüntüsü + mesaj geçmişi modele gönderilir
5. Model prediction döndürür, parseAction() ile parse edilir
6. Her parsed action için operator.execute() çağrılır
7. finished veya call_user gelirse döngü biter
8. Hata veya maxLoop aşılırsa döngü biter

2.4. UI-TARS ile VILAG Arasındaki Farklar

> *Önemli: Kılavuz VILAG kod tabanına göre yazılmıştır. UI-TARS Desktop referans amaçlı kullanılabilir ancak kod yazılırken VILAG'ın yapısı esas alınacaktır.*

Özellik	UI-TARS Desktop	VILAG
Namespace	@ui-tars/*	@vilag/*
Model API	Chat Completions + Response API	Sadece Chat Completions
Retry	asyncRetry kütüphanesi	Manuel retry döngüsü
pause/resume	Promise tabanlı tam mekanizma	Flag tabanlı, eksik (eklenecek)
Context	useContext pattern	Yok, doğrudan erişim
Logger	console	Özel logger (renkli, timestamplı)
Mesaj Yapısı	Conversation (from/value)	Message (role/content) - OpenAI uyumlu
Action Parser	Harici @ui-tars/action-parser	Özel @vilag/action-parser

2.5. Kritik Entegrasyon Noktaları

- **System Prompt (GUIAgent.ts constructor):** RAG'dan çekilen bilgiyi buraya enjekte edeceğiz
- **run() metodu (GUIAgent.ts):** Planner'ı run() öncesinde, HITL'ı execute() öncesinde çağıracağız
- **emitData callback:** HITL için kullanıcıya bilgi iletimi burada olacak
- **main.ts (Electron):** Agent'ı başlatan ve IPC üzerinden UI ile iletişimi sağlayan dosya

3. Geliştirme Yol Haritası

3.1. Geliştirme Sırası

Sıra	Modül	Süre	Bağımlılık
0	pause/resume mekanizması	2-3 gün	Yok
1	RAG Sistemi	2-3 hafta	Yok
2	Planner Katmanı	2-3 hafta	RAG
3	Human-in-the-Loop	1-2 hafta	pause/resume + Planner
4	Entegrasyon ve Test	2 hafta	Hepsi

4. Ön Koşul: pause/resume Mekanizması

> *VILAG'ın GUIAgent'ında isPaused ve isStopped flag'leri var ama UI-TARS'taki gibi Promise tabanlı bekleme mekanizması eksik. Human-in-the-Loop için bu şart.*

4.1. Neden Gerekli?

HITL'de agent bir aksiyonu uygulamadan önce durup kullanıcıdan onay beklemesi gerekiyor. Sadece flag kontrolü yeterli değil çünkü döngü flag'i kontrol ettikten sonra devam eder. Bir Promise ile döngüyü gerçekten durdurmamız lazım.

4.2. Eklenecek Kod

GUIAgent.ts'e şu değişiklikler yapılacak:

4.2.1. Yeni değişkenler

```
// GUIAgent sınıfına ekle:
private resumePromise: Promise<void> | null = null;
private resolveResume: (() => void) | null = null;
```

4.2.2. pause() metodu

```
public pause(): void {
  this.isPaused = true;
  this.resumePromise = new Promise((resolve) => {
    this.resolveResume = resolve;
  });
  this.logger.info('[GUIAgent] Paused');
}
```

4.2.3. resume() metodu

```
public resume(): void {
  if (this.resolveResume) {
    this.resolveResume();
    this.resumePromise = null;
    this.resolveResume = null;
  }
  this.isPaused = false;
  this.logger.info('[GUIAgent] Resumed');
}
```

4.2.4. run() içinde kullanım

```
// Döngünün başında, stop/abort kontrollerinden önce:
if (this.isPaused && this.resumePromise) {
  this.logger.info('[GUIAgent] Waiting for resume...');
  await this.resumePromise;
}
```

> *Bu değişiklik küçük ama kritik. HITL modülü tamamen buna bağlı. Önce bunu yapıp test edin.*

5. Modül 1: RAG (Retrieval-Augmented Generation)

5.1. Amaç

Modele Teams'in bilinen iş akışlarını referans olarak sunarak doğru aksiyonları daha tutarlı üretmesini sağlamak. Model sıfırdan tahmin etmek yerine veritabanındaki adım adım talimatları referans alacak.

5.2. Dosya Yapısı

```
packages/
  vilag-rag/                    → Yeni paket
    package.json
    src/
      index.ts                  → Ana export
      ScenarioStore.ts           → Senaryoları yükler ve saklar
      Retriever.ts               → Benzerlik araması yapar
      PromptInjector.ts           → Senaryoyu system prompt'a ekler
      types.ts                   → RAG tip tanımları
    scenarios/
      teams/
        create-meeting.json
        send-message.json
        share-file.json
        ...
```

5.3. package.json

```
{
  "name": "@vilag/rag",
  "version": "0.0.1",
  "dependencies": {
    "@vilag/shared": "workspace:*"
  }
}
```

> *pnpm-workspace.yaml dosyasına packages/vilag-rag yolunu eklemeyi unutmayın.*

5.4. Senaryo Veri Yapısı

```
// types.ts
export interface ScenarioStep {
  order: number;
  action: string;        // "Sol panelde Calendar ikonuna tıkla"
  element?: string;      // "Calendar icon"
  actionType?: string;   // "click" | "type" | "scroll"
}

export interface Scenario {
  id: string;
  title: string;
  description: string;
  keywords: string[];
  steps: ScenarioStep[];
  preconditions: string[];
  expectedResult: string;
}
```

5.5. Retrieval Stratejisi

Başlangıçta keyword matching ile başlayın. Kullanıcının komutundaki kelimeleri senaryoların keywords dizisiyle eşleştirin. En çok eşleşen senaryo seçilir.

```
// Retriever.ts - basitleştirilmiş örnek
retrieve(query: string): Scenario | null {
  const queryWords = query.toLowerCase().split(/\s+/);
  let bestMatch: Scenario | null = null;
  let bestScore = 0;

  for (const scenario of this.scenarios) {
    const score = scenario.keywords.filter(
      kw => queryWords.some(w => w.includes(kw) || kw.includes(w))
    ).length;
    if (score > bestScore) {
      bestScore = score;
      bestMatch = scenario;
    }
  }
  return bestScore > 0 ? bestMatch : null;
}
```

> *İleride embedding tabanlı benzerlik aramasına geçebilirsiniz (OpenAI text-embedding-ada-002 veya sentence-transformers). Ama başlangıç için keyword matching yeterli.*

5.6. System Prompt'a Enjeksiyon

VILAG'ın GUIAgent'ında system prompt, constructor'da config üzerinden veya run() başlangıcında mesaj dizisine ekleniyor. RAG'dan gelen senaryo, system prompt'un sonuna eklenir:

```
// PromptInjector.ts
export function injectScenario(
  basePrompt: string,
  scenario: Scenario | null
): string {
  if (!scenario) return basePrompt;

  const stepsText = scenario.steps
    .map(s => `${s.order}. ${s.action}`)
    .join('\n');

  return basePrompt + '\n\n' +
    '## Reference Scenario: ' + scenario.title + '\n' +
    stepsText + '\n';
}
```

5.7. GUIAgent Entegrasyonu

GUIAgent.ts'in run() metodunun başında RAG çağrısı yapılır:

```
// run() metodunun başında:
const scenario = this.rag?.retrieve(instruction) ?? null;
const systemPrompt = injectScenario(
  this.config.systemPrompt || DEFAULT_SYSTEM_PROMPT,
  scenario
);
```

```
// Bu systemPrompt mesaj dizisinin ilk elemanı olarak kullanılır
```

5.8. Uygulama Adımları

1. packages/vilag-rag/ dizinini ve package.json'ı oluşturun
2. types.ts: Scenario, ScenarioStep tiplerini tanımlayın
3. scenarios/teams/ altına en az 10-15 JSON senaryo dosyası yazın
4. ScenarioStore.ts: JSON dosyalarını yükleyen ve bellekte tutan sınıfı yazın
5. Retriever.ts: Keyword matching ile en uygun senaryoyu bulan sınıfı yazın
6. PromptInjector.ts: Senaryoyu system prompt'a enjekte eden fonksiyonu yazın
7. GUIAgentConfig'e opsiyonel rag alanı ekleyin
8. GUIAgent.ts run() başında RAG çağrısını entegre edin
9. Test: Farklı komutlarla doğru senaryonun çekildiğini doğrulayın

6. Modül 2: Planner (Planlama Katmanı)

6.1. Amaç

Kullanıcının karmaşık komutlarını alt görevlere bölmek. VILAG'ın GUIAgent'ı tek bir instruction ile çalışıyor. Planner, "Ahmet'e yarın saat 3'te toplantı daveti gönder" gibi bir komutu mantıksal alt adımlara ayırır.

6.2. Dosya Yapısı

```
packages/
  vilag-planner/                  → Yeni paket
    package.json
    src/
      index.ts                    → Ana export
      Planner.ts                   → Komutu alt görevlere bölen sınıf
      PlanExecutor.ts               → Planı adım adım yürüten sınıf
      types.ts                     → Plan, Subtask tipleri
      prompts.ts                   → Planlama LLM prompt'ları
```

6.3. Plan Veri Yapısı

```
// types.ts
export type RiskLevel = 'low' | 'medium' | 'high';

export interface Subtask {
  id: number;
  instruction: string;
  riskLevel: RiskLevel;
  requiresApproval: boolean;
}

export interface Plan {
  originalInstruction: string;
  subtasks: Subtask[];
}
```

6.4. Planner Sınıfı

Planner, ayrı bir LLM çağrısı ile çalışır. VILAG'ın UITarsModel sınıfını yeniden kullanabilirsiniz çünkü zaten OpenAI uyumlu:

```
// Planner.ts
export class Planner {
  private client: OpenAI;

  constructor(config: { baseURL: string; apiKey: string; model: string }) {
    this.client = new OpenAI({
      baseURL: config.baseURL,
      apiKey: config.apiKey,
    });
  }

  async createPlan(instruction: string, scenario?: Scenario): Promise<Plan>
{
    const response = await this.client.chat.completions.create({
```

```
      model: this.config.model,
      messages: [
        { role: 'system', content: PLANNER_SYSTEM_PROMPT },
        { role: 'user', content: instruction +
          (scenario ? '\nReference: ' + JSON.stringify(scenario.steps) : '')
}
      ],
      response_format: { type: 'json_object' },
    });
    return JSON.parse(response.choices[0].message.content);
  }
}
```

6.5. Risk Seviyeleri

Seviye	Açıklama	Örnekler	Onay
low	Geri dönüşü kolay, navigasyon	Menü açma, sayfa geçişi	Hayır
medium	Veri girişi	Başlık yazma, tarih seçme	Hayır
high	Geri dönüşü zor/imkansız	Mesaj gönderme, davet, silme	Evet

6.6. PlanExecutor

PlanExecutor, subtask listesini sırayla GUIAgent.run() ile yürütür. Her subtask için RAG'dan ilgili senaryo çekilir:

```
// PlanExecutor.ts
export class PlanExecutor {
  async executePlan(
    plan: Plan,
    guiAgent: GUIAgent<any>,
    rag: Retriever,
    approvalManager?: ApprovalManager
  ): Promise<void> {
    for (const subtask of plan.subtasks) {
      // 1. RAG'dan senaryo çek
      const scenario = rag.retrieve(subtask.instruction);

      // 2. Onay gerekiyorsa iste
      if (subtask.requiresApproval && approvalManager) {
        const approved = await approvalManager.request(subtask);
        if (!approved) continue;
      }

      // 3. Agent'ı çalıştır
      await guiAgent.run(subtask.instruction);
    }
  }
}
```

6.7. Uygulama Adımları

1. packages/vilag-planner/ dizinini ve package.json’ı oluşturun
2. types.ts: Plan, Subtask, RiskLevel tiplerini tanımlayın
3. prompts.ts: PLANNER_SYSTEM_PROMPT’u yazın
4. Planner.ts: LLM çağrısı ile plan üreten sınıfı yazın
5. PlanExecutor.ts: Subtask’ları sırayla yürüten sınıfı yazın
6. RAG entegrasyonu: Her subtask için senaryo çekilsin
7. Test: Farklı karmaşıklıkta komutlarla planları kontrol edin

7. Modül 3: Human-in-the-Loop

7.1. Amaç

Geri dönüşü zor aksiyonlardan önce kullanıcıdan onay almak. Agent'ın otonom çalışmasını korurken kritik anlarda insan kontrolü sağlamak.

7.2. Dosya Yapısı

```
packages/
  vilag-hitl/                    → Yeni paket
    package.json
    src/
      index.ts
      ApprovalManager.ts          → Onay mantığını yönetir
      types.ts                    → ApprovalRequest/Response tipleri
```

7.3. ApprovalManager

```
// ApprovalManager.ts
export class ApprovalManager {
  private pendingResolve: ((approved: boolean) => void) | null = null;

  constructor(
    private onApprovalRequest: (request: ApprovalRequest) => void
  ) {}

  async request(subtask: Subtask): Promise<boolean> {
    return new Promise((resolve) => {
      this.pendingResolve = resolve;
      this.onApprovalRequest({
        subtaskId: subtask.id,
        description: subtask.instruction,
        riskLevel: subtask.riskLevel,
      });
    });
  }

  approve(): void {
    this.pendingResolve?.(true);
    this.pendingResolve = null;
  }

  reject(): void {
    this.pendingResolve?.(false);
    this.pendingResolve = null;
  }
}
```

7.4. Electron IPC Entegrasyonu

Onay istekleri Electron'ın IPC sistemi üzerinden UI'a iletilir:

```
// apps/vilag-desktop/src/main/main.ts içinde:
const approvalManager = new ApprovalManager((request) => {
  // UI'a onay isteği gönder
```

```
  mainWindow?.webContents.send('approval-request', request);
});

// UI'dan gelen cevabı dinle
ipcMain.on('approval-response', (_event, approved: boolean) => {
  if (approved) approvalManager.approve();
  else approvalManager.reject();
});
```

7.5. React UI Tarafı

Renderer'da bir onay diyaloğu gösterilir:

```
// src/renderer/src/components/ApprovalDialog.tsx
// window.vilagAPI.onApprovalRequest((request) => { ... })
// Kullanıcı Onayla/Reddet butonuna tıklar
// window.vilagAPI.respondApproval(true/false)
```

7.6. Uygulama Adımları

1. packages/vilag-hitl/ dizinini ve package.json'ı oluşturun
2. types.ts: ApprovalRequest, ApprovalResponse tiplerini tanımlayın
3. ApprovalManager.ts: Promise tabanlı onay yöneticisini yazın
4. vilag-electron-ipc'ye approval kanallarını ekleyin
5. main.ts'de IPC dinleyicilerini kurun
6. React'ta ApprovalDialog bileşenini oluşturun
7. PlanExecutor'da requiresApproval kontrolünü entegre edin
8. Test: High-risk senaryoda agent'ın durduğunu doğrulayın

8. Modüllerin Entegrasyonu

8.1. Uçtan Uca Akış

1. Kullanıcı komut verir: “Ahmet’e yarın 15:00’te toplantı daveti gönder”
2. Planner komutu alt görevlere böler (4 subtask)
3. PlanExecutor ilk subtask’ı alır: “Calendar bölümüne git”
4. RAG bu subtask için ilgili senaryoyu çeker
5. Senaryo system prompt’a enjekte edilir
6. requiresApproval = false → GUIAgent.run() doğrudan çalışır
7. Subtask tamamlanır, sonraki subtask’a geçilir
8. Son subtask (davet gönder): requiresApproval = true
9. ApprovalManager devreye girer, UI’da onay diyaloğu gösterilir
10. Kullanıcı onaylar, agent daveti gönderir

8.2. Ana Orkestrasyon Kodu

apps/vilag-desktop/src/main/main.ts içinde IPC üzerinden tetiklenen ana akış:

```
async function handleUserCommand(instruction: string) {
  // 1. Plan oluştur
  const plan = await planner.createPlan(instruction);

  // 2. Her subtask için
  for (const subtask of plan.subtasks) {
    // 3. RAG’dan senaryo çek
    const scenario = rag.retrieve(subtask.instruction);

    // 4. System prompt’u güncelle
    const prompt = injectScenario(basePrompt, scenario);

    // 5. Onay gerekiyorsa iste
    if (subtask.requiresApproval) {
      const approved = await approvalManager.request(subtask);
      if (!approved) {
        logger.info(`Subtask ${subtask.id} rejected by user`);
        continue;
      }
    }

    // 6. GUIAgent ile yürüt
    const agent = new GUIAgent({
      operator: currentOperator,
      model: modelConfig,
      systemPrompt: prompt,
      signal: abortController.signal,
      onData: ({ data }) => {
        mainWindow?.webContents.send('agent-data', data);
      },
    });
    await agent.run(subtask.instruction);
  }
```

```
}
```

8.3. Hata Senaryoları

- **Subtask başarısız olursa:** 2-3 retry yapın, hala başarısızsa kullanıcıya bilgi verip durdurun.
- **RAG senaryo bulamazsa:** Sorun değil, model mevcut davranışıyla devam etsin. RAG opsiyonel bir iyileştirme.
- **Planner hatalı plan üretirse:** JSON parse hatası veya boş subtask durumunda bilgi verin.
- **Kullanıcı onayı reddederse:** O subtask'ı atlayın veya tüm planı iptal etme seçeneği sunun.

9. Teams Senaryo Veritabanı

9.1. Yüksek Öncelikli Senaryolar

Senaryo	Açıklama	Adım
Mesaj gönderme	Bir kişiye chat mesajı	4-5
Toplantı oluşturma	Yeni toplantı planlama	6-8
Toplantıya katılma	Mevcut toplantıya katılma	3-4
Dosya paylaşma	Chat/kanalda dosya paylaşma	5-6
Kanal mesajı	Takım kanalına mesaj	4-5
Kişi arama	Teams'te kişi arama	2-3
Durum değiştirme	Online/busy/away değiştirme	3-4

9.2. Orta Öncelikli Senaryolar

Senaryo	Açıklama	Adım
Grup oluşturma	Yeni grup chat	5-6
Ekran paylaşma	Toplantıda ekran paylaşma	3-4
Tepki ekleme	Mesaja emoji tepkisi	2-3
Dosya indirme	Paylaşılan dosyayı indirme	3-4
Bildirim ayarları	Kanal/chat bildirimleri	4-5
Takvim görüntüleme	Haftalık/günlük takvim	2-3

9.3. Senaryo Yazma Rehberi

- Adımlar atomik olsun: her adım tek bir aksiyon içersin
- Element tanımları açık olsun: "sol paneldeki Calendar ikonu" gibi
- Ön koşulları belirtin: "Teams açık ve kullanıcı giriş yapmış"
- Senaryoları Teams'in gerçek arayüzünde test ederek yazın
- Türkçe ve İngilizce keyword'ler ekleyin (toplantı/meeting gibi)

10. Test Stratejisi

10.1. Birim Testler

- **RAG Retriever:** Farklı komutlarla doğru senaryonun çekildiğini doğrulayın
- **Action Parser:** @vilag/action-parser'ın v1.0 ve v1.5 formatlarını doğru parse ettiğini test edin
- **Planner:** Mock LLM cevapları ile planların doğru parse edildiğini test edin
- **ApprovalManager:** Onay/red durumlarının doğru yönetildiğini test edin

10.2. Metrikler (Tez İçin)

Metrik	Açıklama	Nasıl Ölçülür
Başarı Oranı	Görevin doğru tamamlanma yüzdesi	Manuel test: 20 görev, kaçı başarılı
RAG Etkisi	RAG ile/RAG'sız başarı oranı farkı	A/B karşılaştırma
Planner Doğruluğu	Üretilen planların mantıksal doğruluğu	Manuel değerlendirme
HITL Müdahale Oranı	Kullanıcının onay/red oranı	Logger'dan otomatik
Ortalama Adım Sayısı	Görev başına aksiyon sayısı	Logger'dan otomatik

11. Git Workflow

11.1. Branch Yapısı

```
main                ← Stabil sürüm
dev                 ← Ana geliştirme branch'i
feature/pause-resume   ← pause/resume mekanizması
feature/rag            ← RAG modülü
feature/planner        ← Planner modülü
feature/hitl           ← Human-in-the-Loop
```

11.2. Merge Sırası

1. feature/pause-resume → dev
2. feature/rag → dev
3. feature/planner → dev
4. feature/hitl → dev

11.3. Commit Mesajları

```
feat(rag): add ScenarioStore with JSON loading
feat(rag): implement keyword-based retrieval
feat(planner): add Planner class with LLM integration
feat(hitl): add ApprovalManager with pause/resume
fix(rag): handle empty scenario results gracefully
test(rag): add retriever unit tests
```

12. Zaman Planlaması

12.1. Haftalık Plan

Hafta	Görev	Çıktı
1	pause/resume + RAG: Senaryolar	Mekanizma hazır, 10-15 senaryo yazılmış
2-3	RAG: ScenarioStore + Retriever + PromptInjector	Çalışan retrieval + enjeksiyon
4-5	Planner: Planner + PlanExecutor	Komut → alt görev bölme + yürütme
6	Planner + RAG entegrasyonu	Her subtask için RAG çalışıyor
7-8	Human-in-the-Loop + IPC + UI	Onay mekanizması çalışıyor
9-10	Entegrasyon testleri + manuel testler	Uçtan uca akış test edilmiş
11-12	Metrik toplama + tez yazımı	Test sonuçları, metrikler

12.2. Minimum Viable Product (MVP)

Zaman daralırsa şu minimum kapsamı hedefleyin:

- 7-10 temel Teams senaryosu (keyword matching ile RAG)
- Basit bir Planner (2-3 alt göreve bölme)
- Sadece high-risk adımlarda basit onay (evet/hayır)
- En az 5 farklı komutla test edilmiş çalışan demo

> *MVP'yi erken çıkarın, sonra iyileştirin. Çalışan basit bir sistem, çalışmayan karmaşık bir sistemden her zaman iyidir.*